Application Artefacts Structure & Runtime Guide

Drishti IAS

17 March 2026

Application Artefacts Structure & Runtime Guide

This document categorizes the files and directories in the `inboxpilot-agentic-ai` repository into the four defined artifact categories for an Agentic AI platform: **Models**, **Tools**, **Agents**, and **Orchestration**. For each category, we define the Development Model, Source Artefacts, Runtime environments, Dockerization approaches, and setup instructions.

1. Models

Models encompass the formal list of foundational models, parameter configurations, and data schemas required for the system's reasoning and operation.

Development Model & Source Artefacts:

- Developed as data schemas, configuration sets, and object relational mappings. These statically define how the fundamental data should be structured, validated, and loaded.
- `config.yaml`: Contains the static configurations and parameters required for the foundational models and overall system variables.
- `inboxpilot/config_loader.py`: Loads the `config.yaml` configuration, interpreting hyperparameters and global variables for the models.
- `inboxpilot/models.py`: Contains the fundamental data schemas (e.g., Pydantic models) used by the application to structure the input and output flowing through foundational models.
- `a2a/models.py`: Defines the specific data schemas and formats for agent-to-agent communication payloads.
- `inboxpilot/state.py`: Defines the schema for the application state, acting as the memory structures required by the system's reasoning operations to maintain context across steps.
- `inboxpilot/db.py`: Provides data schema implementations or database interactions necessary to store and retrieve stateful data used by the models and agents.

Runtime for the Artefact:

- In this application, models refer to the structural schemas and configurations defined in python which are loaded via `config.yaml` and initialized across the respective docker containers as memory/state references.
- These components are loaded natively in the active Python environment across all microservices and are injected into inference API calls.

Dockerization & Runtime Setup:

- Models do not have independent Dockerfiles in this structure but are built upon via `Dockerfile.base`, which packages the pydantic schemas and logic definitions so that any downstream container has access to them in its runtime.

2. Tools

Tools form the capability dictionary for the overall system, enabling agents to interact with external APIs and internal microservices.

Development Model & Source Artefacts:

- Tools operate as discrete Python modules using framework bindings to map local functions to executable instructions compatible with LLM function calling.
- **`inboxpilot/tools/` (Directory: `calendar_tool.py`, `draft_tool.py`, `imap_tool.py`, `search_tool.py`, `style_tool.py`, `__init__.py`)**: This entire directory acts as the registry for external interactions. Each file defines specific capabilities. These provide actionable programmatic interfaces (functions) for the language models to use.

Runtime for the Artefact:

- Executed as live, addressable capabilities directly within an agent's process. The execution flow requires structured parameters generated by an LLM to invoke the python capability, which queries external APIs and returns the JSON payload back to the overarching graph.

Dockerization & Runtime Setup:

- Tools are executed within the Docker environments of the Agents orchestrating them. As they are heavily dependent on external APIs, configuration relies on environment variables passed to the parent docker containers.

3. Agents

Agents are the specialized workers that execute logic, maintain context, and dispatch specific tasks to tools or models depending on the routed logic.

Development Model & Source Artefacts:

- Agents are developed as individual python environments executing pre-determined reasoning paths. They use specific prompt designs, stateful definitions, and graph/flow bindings (e.g., LangGraph).
- **`inboxpilot/agents/` (Directory: `classify_agent.py`, `fetch_agent.py`, `respond_agent.py`, `__init__.py`)**: Contains the logical blueprints for the specific workers in the system.
- **`scripts/start_classify.py`, `scripts/start_fetch.py`, `scripts/start_respond.py`**: Bootstrapping scripts that spin up individual agent runtimes.

Runtime for the Artefact:

- Agents act as live executing microservices that process structured inputs, evaluate local graphs against LLMs, utilize tools, and return an output state to an overarching orchestrator.
- They rely on FastAPI or standard socket loops depending on the entry method and serve as live endpoints.

Dockerization:

- **`services/classify-service/Dockerfile`**: Containerizes the classification agent.
- **`services/fetch-service/Dockerfile`**: Containerizes the fetching agent.
- **`services/respond-service/Dockerfile`**: Containerizes the responding agent.
- These images are built dynamically using `Dockerfile.base` as an underpinning environment.

Runtime Setup:

To build and run an individual agent service:

```
# Build the base image
docker build -t inboxpilot-base -f Dockerfile.base .

# Build and run the specific service
cd services/classify-service
docker build -t classify-agent .
docker run -p 8001:8000 classify-agent
```

4. Orchestration

Orchestration handles the overarching control logic, directing workflows, coordinating agents, defining deployment infrastructure, and managing the event flow.

Development Model & Source Artefacts:

- Built on directed acyclic graph formats and execution loops tracking states and distributing data between separate microservices.
- **`inboxpilot/orchestrator.py`** & **`main.py`**: Contains the core execution workflow and event loops.
- **`services/supervisor/`**: Contains the core routing logic (`graph.py`, `main.py`, `cli.py`) that governs the DAG.
- **`a2a/`**: Networking handlers (`client.py`, `server.py`) that govern decentralized agent-to-agent communication.
- **`inboxpilot/cli.py`**: Command-line interfaces for execution trigger.
- **`scripts/start_supervisor.py`, `scripts/run_all_services.py`**: High-level bootstrapping.

Runtime for the Artefact:

- Orchestration functions as a master supervisor loop actively routing messages and coordinating interactions across the infrastructure. It communicates over standard network layers to the independent microservice endpoints of the child agents.

Dockerization:

- **`docker-compose.yml`**: Defines the execution constraints and networking fabric for all containers simultaneously.
- **`Dockerfile.base`**: Sets the global environment foundation.
- **`services/supervisor/Dockerfile`**: Containerizes the master orchestration instance.

Runtime Setup:

To initialize the entire orchestration environment including all agents:

```
# Run everything mapped in docker-compose
docker-compose up --build
```